Agentic AI for Optimizing FAISS HNSW Vector Database Parameters

Ryan Zhao

December 11, 2025

Abstract

Hierarchical Navigable Small World (HNSW) graphs are widely used for high-performance approximate nearest neighbor search in vector databases, but their performance depends on three coupled parameters (M, efConstruction, efSearch) that are difficult to tune by hand. We present an *agentic AI system* that uses a language model to propose FAISS HNSW configurations, evaluate them, and refine its strategy using logged experiments. The agent supports two workload profiles: a Knowledge + Reasoning database optimized for high recall (target ≥ 0.90) and a Memory + Reaction database optimized for low latency while maintaining recall around 0.70. On million-vector synthetic datasets, the system reliably finds acceptable configurations for both profiles within a small number of iterations, reducing manual tuning effort significantly.

1 Introduction

Vector databases have become essential infrastructure for modern AI applications, enabling efficient similarity search across high-dimensional embeddings for tasks such as retrieval-augmented generation (RAG), recommendation systems, and semantic search. Among various indexing techniques, Hierarchical Navigable Small World (HNSW) graphs [MY18] have emerged as a leading solution, offering an excellent balance between search accuracy and query latency.

FAISS (Facebook AI Similarity Search) [JDJ17, Res24] provides a high-performance implementation of HNSW, but achieving optimal performance requires careful tuning of three interdependent parameters:

- **M:** Graph connectivity, controlling the number of outgoing edges per node. Higher values improve recall but increase memory usage and build time.
- **efConstruction:** Beam width during index construction. Higher values produce better graph quality and recall, but significantly increase build time.
- **efSearch:** Beam width during query execution. Higher values improve recall but increase query latency.

The parameter space is high-dimensional and non-linear, with complex trade-offs between recall, latency, and memory. Traditional approaches rely on fixed heuristics, grid search, or manual expert tuning, all of which are computationally expensive, inflexible, and often fail to find optimal configurations for specific use cases.

This research introduces an **agentic AI system** that autonomously optimizes FAISS HNSW parameters through iterative experimentation and reasoning. Unlike static optimization methods, our agent adapts its strategy based on performance feedback, learns from historical experiments, and can optimize for different objectives depending on the application requirements.

2 Background and Related Work

HNSW (Hierarchical Navigable Small World) graphs [MY18] construct a multi-layer graph that supports logarithmic-time approximate nearest neighbor search. Their performance is dependent on three

parameters (M, efConstruction, efSearch), which jointly control recall, latency, and memory footprint. FAISS [JDJ17, Res24] provides optimized CPU and GPU implementations of HNSW but offers little guidance on how to tune these parameters for particular workloads.

Automated hyperparameter optimization has been widely studied for machine learning models, using Bayesian optimization, evolutionary search, and reinforcement learning. In contrast, relatively few works target low-level database index parameters, and most still rely on black-box search. Recent advances in large language models (LLMs) have enabled *agentic AI systems* that can reason about objectives and constraints, making them a natural fit for steering such search processes rather than replacing them.

3 Methodology

Our agentic optimization system, implemented using LangGraph, operates as a stateful workflow. At each iteration the system (i) analyzes the dataset, (ii) proposes parameters, (iii) builds and evaluates an index, and (iv) decides whether and how to continue. Dataset analysis records size and dimensionality; parameter generation uses an LLM; evaluation measures recall, latency, and memory; and decision logic updates the best configuration and stops once a maximum number of experiments or performance targets are reached.

The optimization is structured into two phases. In the *recall phase*, the agent explores configurations that drive recall above a minimum workload-specific threshold (e.g., ≥ 0.90 for Knowledge + Reasoning and around 0.70 for Memory + Reaction). Once that target is met, it switches to a *latency phase* where it tries to reduce latency (primarily by lowering efSearch) while keeping recall above the same threshold. If recall degrades, the agent automatically returns to the recall phase.

We capture workload differences via simple database "profiles". The Knowledge + Reasoning profile assumes that missing relevant results is costly and therefore prefers higher M and efSearch and is allowed higher latency and memory usage. The Memory + Reaction profile assumes tight latency and memory budgets, prefers smaller M and efSearch, and treats recall thresholds as minimal acceptability constraints rather than primary objectives.

The agent uses LLM-guided parameter proposals that learn from both recent trials in the current run and historical experiments from past runs. To optimize for expensive large-scale experiments, we eliminated the initial exploration phase by default and implemented index reuse between evaluation steps, reducing build time by approximately 50% for million-scale datasets. All trials are logged to JSONL files organized by database type and dataset size for cross-run learning and analysis.

4 Experiments and Results

4.1 Experimental Setup

Experiments were conducted using:

- FAISS version 1.7.4
- Synthetic datasets: 1M-vector scale, 128 dimension
- Query sets: 10,000 queries per dataset
- Evaluation metrics: Recall@10, true recall (brute-force for datasets up to 1M vectors), average query latency (ms per query), memory (GB), index size (MB)
- Optimization procedure: 5 iterations per experiment for each workload profile. We selected five iterations as preliminary experiments showed that performance metrics converged within this range for both workload profiles.

We compute Recall@10 using ground-truth nearest neighbors obtained via brute-force search. Throughout the paper we refer to this quantity as "true recall" to distinguish it from proxy estimates.

4.2 Results on Million-Scale Datasets

We report results on synthetic 128-dimensional datasets with 10^6 vectors for both Knowledge + Reasoning and Memory + Reaction workloads. All metrics are computed using exact brute-force recall computation (IndexFlatL2) for accurate evaluation. The agent successfully achieved target performance metrics for both workload profiles through iterative optimization.

Knowledge + Reasoning Profile The agent optimized for high recall (target ≥ 0.90) and achieved excellent results. Across multiple optimization runs, the system consistently found configurations that met or exceeded the recall target. The following three configurations are representative examples drawn from the 50 experiments (5 iterations each):

- Configuration ($M = 40$, efConstruction $= 362$, efSearch $= 681$): True recall@10 of 0.998 with latency 0.23 ms
- Configuration ($M = 32$, efConstruction $= 400$, efSearch $= 200$): True recall@10 of 0.955 with latency 0.31 ms
- Configuration ($M = 32$, efConstruction $= 425$, efSearch $= 370$): True recall@10 of 0.901 with latency 0.14 ms

These results illustrate that the agent consistently identified configurations surpassing the 0.90 recall requirement for the Knowledge + Reasoning profile. Several runs achieved near-perfect recall with sub-millisecond latency, such as the configuration ($M = 40$, efConstruction $= 362$, efSearch $= 681$), which reflects the agent's tendency to select higher graph connectivity and sufficiently deep search when optimizing for accuracy-oriented workloads.

Memory + Reaction Profile The agent optimized for low latency while maintaining recall around 0.70. Representative results from our 50 experiments (5 iterations each) include:

- Configuration ($M = 24$, efConstruction $= 200$, efSearch $= 50$): True recall@10 of 0.691 with latency 0.065 ms
- Configuration ($M = 16$, efConstruction $= 300$, efSearch $= 100$): True recall@10 of 0.690 with latency 0.085 ms
- Configuration ($M = 16$, efConstruction $= 300$, efSearch $= 120$): True recall@10 of 0.7195 with latency 0.096 ms

The agent also produced configurations that balanced accuracy and speed for the Memory + Reaction profile. For example, the configuration ($M = 24$, efConstruction $= 200$, efSearch $= 50$) achieved a recall of 0.691 with a latency of 0.065 ms, illustrating how the system leverages moderate connectivity and minimal search depth to prioritize responsiveness while maintaining acceptable recall.

4.3 Summary of Workload-Specific Tuning

Table 1 summarizes the key quantitative results for the two workloads on the million-scale dataset we evaluated.

Workload	**Dataset Size**	**Avg Recall@10**	**Avg Latency (ms)**
Knowledge + Reasoning	10^6	0.924	0.313
Memory + Reaction	10^6	0.693	0.073

Table 1: Average recall and latency across 50 experiments (5 iterations each) for both workload profiles on the million-scale dataset.

The results in Table 1 illustrate clear and consistent distinctions between the two workload profiles. The Knowledge + Reasoning profile exhibits a strong bias toward high-recall configurations, with many runs approaching near-perfect accuracy. In contrast, the Memory + Reaction profile consistently favors lightweight graph structures and minimal search depth, resulting in sub-millisecond latency

while maintaining recall near the 0.70 target. The agent converges reliably across all runs, adapting its parameter choices to the qualitative priorities of each workload rather than gravitating toward a single global optimum. This divergence in selected configurations highlights the system's ability to internalize and optimize for different operational objectives.

5 Discussion

This work demonstrates that agentic systems can guide the tuning of low-level vector database parameters by combining structured workflows with LLM-based reasoning. Through a phase-aware exploration strategy and lightweight workload profiles, the agent adapts its search behavior to different optimization objectives and discovers distinct parameter configurations for reasoning-based and reaction-based workloads.

Several limitations constrain practical deployment. Large-scale experiments remain computationally expensive because each iteration requires rebuilding an HNSW index and issuing thousands of queries. This restricts the amount of feasible exploration for datasets beyond million scale and for applications that require frequent re-tuning. A central bottleneck arises from the cost of computing ground-truth neighbors through brute-force search. While this approach is manageable in controlled research environments, it becomes prohibitive for datasets with tens or hundreds of millions of vectors or for systems that require continuous monitoring and adaptation. Without access to true recall, the agent must rely on approximate feedback signals that reduce evaluation accuracy.

Several strategies can help mitigate this limitation. Ground-truth neighbors can be cached for stable portions of the dataset. Recall can be estimated using a representative subset of queries. High-`efSearch` configurations can act as proxy ground truth when full brute-force evaluation is infeasible. Another direction is the development of learned recall estimators that predict recall from inexpensive features of the search process, including distance distributions, candidate list behavior, and search depth. These options reduce computational cost while preserving useful optimization signals.

Another limitation of this study is the use of synthetic datasets, which provide controlled and scalable conditions but do not capture the full variability of real-world embedding distributions. In practical applications, vector databases may exhibit domain-specific structure, uneven density, or multimodal properties that influence optimal HNSW parameters. Evaluating the agent on text, image, or multimodal embeddings would provide a clearer understanding of how well the approach generalizes to real-world retrieval workloads.

Future work may extend the agent framework in several directions. Reinforcement learning could be used to learn policies that manage exploration and exploitation more effectively and reveal non-obvious parameter interactions. Multi-agent architectures may allow specialization across different phases or workloads. Broader FAISS index support, GPU-accelerated evaluation pipelines, and real-time workload monitoring present additional opportunities for improving scalability and responsiveness in practical systems.

6 Conclusion

We developed an agentic optimization framework that tunes FAISS HNSW parameters through iterative experimentation guided by language models. By structuring optimization into recall-oriented and latency-oriented phases and by defining simple workload profiles, the system identifies effective configurations for both Knowledge + Reasoning and Memory + Reaction databases. On million-vector datasets, the agent achieves recall of at least 0.90 for the Knowledge + Reasoning profile and sub-millisecond latency with recall near 0.70 for the Memory + Reaction profile across 50 experiments with 5 iterations each.

These findings show that even lightweight agentic workflows can improve HNSW parameter configurations while adapting to different operational objectives. The modular separation of measurement, reasoning, and decision logic supports cross-run learning and makes the framework extensible to additional FAISS index types and real-world retrieval workloads.

Exact ground-truth evaluation remains a practical bottleneck at large scale. Sampling-based evaluation, proxy recall estimates, and learned recall predictors provide promising alternatives that reduce

computational cost while maintaining reliable feedback. Incorporating such methods may allow future systems to achieve continuous workload-aware tuning for large and dynamic vector databases.

References

[JDJ17] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus, 2017.

[MY18] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, 2018.

[Res24] Facebook AI Research. Faiss library. https://github.com/facebookresearch/faiss, 2024. Accessed: 2025-11-10.